

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Medium (Scenario Based Assessment)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO2: *Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)*

QUESTIONS: 5

Total Marks: 100

ANNEXURE A

Scenario Based Assessment

Code of Conduct:

I N.Adi Narayana(192525346) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Likhitha

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis				

Application of Concepts				
Analytical & Decision-Making Skills				
Solution Design & Approach				
Clarity, Justification & Presentation				

ANNEXURE

Scenario Based Assessment

Q1: Student Registration System

Suppose that a university system stores student data, where each student must be unique and can enroll in multiple courses.

- Enumerate three Collection interfaces suitable for this system.
- Design a Collection structure (diagram/representation) to store students and their enrolled courses.
- Write a Java program using HashSet and HashMap with Generics to:

- Add students
- Map students to courses
- Display data using Iterator A

suitable structure is:

- Set ➤ Stores unique elements.
 - Useful for storing unique student IDs/names.
 - Example: HashSet<String>
2. Map
 - Stores data in key-value pairs.
 - Useful for mapping a student to their enrolled courses.
 - Example: HashMap<String, HashSet<String>>

List

- Stores elements in an ordered sequence.
- Useful when maintaining the order of courses or students is required.

➤ Example: ArrayList<String>

Java Representation

```
HashSet<String> students;
```

```
HashMap<String, HashSet<String>> studentCourses;
```

For example: students = { "S101", "S102", "S103" }

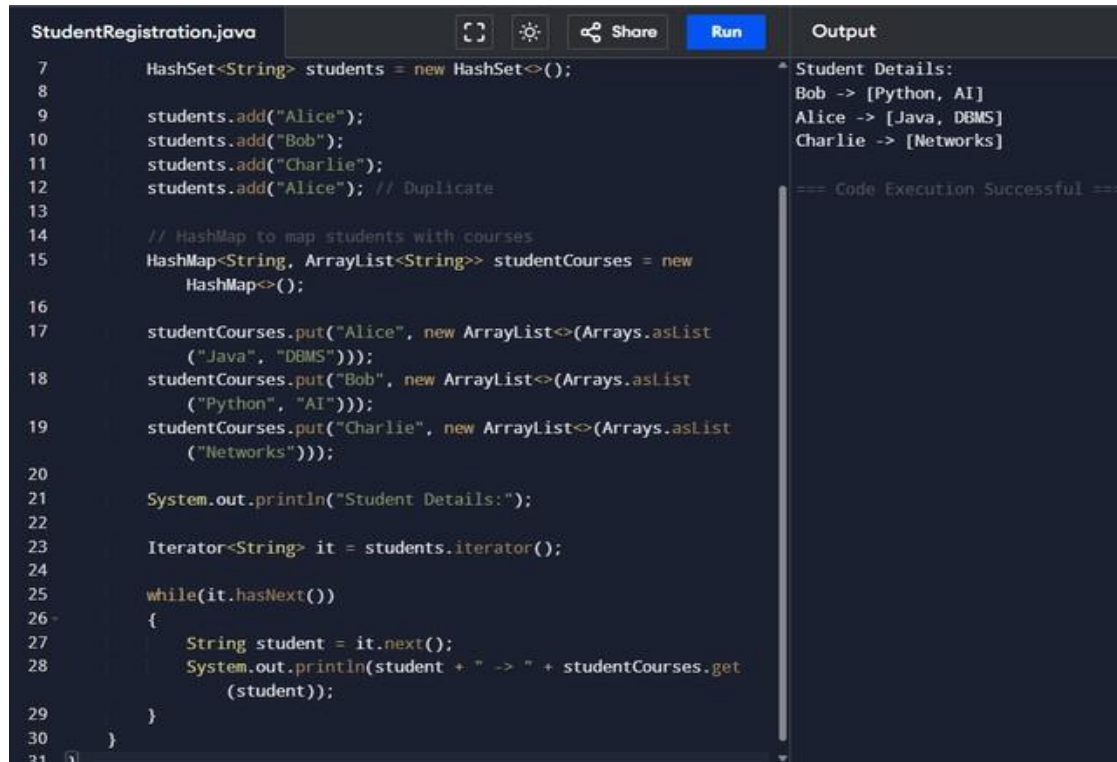
studentCourses = { S101 → [Java, DBMS], S102 → [Python, AI], S103 → [Java, AI] } Here:

- HashSet ensures that each student is unique.

HashMap maps each student ID to their courses.

Inner HashSet ensures that a course is not added twice for the same student.

- Generics provide type safety.



```
StudentRegistration.java
7      HashSet<String> students = new HashSet<>();
8
9      students.add("Alice");
10     students.add("Bob");
11     students.add("Charlie");
12     students.add("Alice"); // Duplicate
13
14     // HashMap to map students with courses
15     HashMap<String, ArrayList<String>> studentCourses = new
        HashMap<>();
16
17     studentCourses.put("Alice", new ArrayList<>(Arrays.asList
        ("Java", "DBMS")));
18     studentCourses.put("Bob", new ArrayList<>(Arrays.asList
        ("Python", "AI")));
19     studentCourses.put("Charlie", new ArrayList<>(Arrays.asList
        ("Networks")));
20
21     System.out.println("Student Details:");
22
23     Iterator<String> it = students.iterator();
24
25     while(it.hasNext())
26     {
27         String student = it.next();
28         System.out.println(student + " -> " + studentCourses.get
            (student));
29     }
30 }
31
```

Output

```
Student Details:
Bob -> [Python, AI]
Alice -> [Java, DBMS]
Charlie -> [Networks]
=== Code Execution Successful ===
```

Q2: E-Commerce Cart System

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

- (a) List three Collection classes applicable for cart management.
- (b) Represent how cart items are stored using a List-based structure.
- (c) Write a Java program using ArrayList and Iterator to:

- Add items
- Remove an item
- Display all items

- (a) Three Advantages of Generics in Collections

1. Type Safety

- o Generics ensure that only the specified type of data is stored in a collection.
- o Example: List<Employee> can store only Employee objects.

3. No Explicit Type Casting

- o When retrieving elements, explicit type casting is not required.
- o This makes the program easier to read and reduces errors.

4. Code Reusability

- o The same collection structure can be used with different data types.
- o Example: List<String>, List<Integer>, List<Employee>

. (b) Representation of Employee Data Using List with Generics

We can use ArrayList<Employee> to store employee objects. Employee Data | List<Employee> | +-----

ID:	101	ID: 102	ID: 103	Name: Ravi	Name: Arun	Name: Priya	Salary: 45000	Salary: 60000	Salary: 75000

Java Representation

```
List<Employee> employees = new ArrayList<>();
```

Example: employees

```
↓ [Employee(101, Ravi, 45000),  
   Employee(102, Arun, 60000),  
   Employee(103, Priya, 75000)]
```

EcommerceCart.java

```
1- import java.util.*;  
2-  
3- public class EcommerceCart {  
4-     public static void main(String[] args) {  
5-  
6-         ArrayList<String> cart = new ArrayList<>();  
7-  
8-         // Add items  
9-         cart.add("Laptop");  
10-        cart.add("Mouse");  
11-        cart.add("Keyboard");  
12-        cart.add("Headphones");  
13-  
14-        // Remove an item  
15-        cart.remove("Mouse");  
16-  
17-        System.out.println("Cart Items:");  
18-  
19-        Iterator<String> it = cart.iterator();  
20-  
21-        while(it.hasNext())  
22-        {  
23-            System.out.println(it.next());  
24-        }  
25-    }  
26-}
```

Output

```
Cart Items:  
Laptop  
Keyboard  
Headphones  
  
=== Code Execution Complete ===
```

Q3: Library Management System

Suppose that a library stores books with unique ISBN numbers and needs fast retrieval.

- (a) Name three Map implementations in Java.
- (b) Design a schema-like representation using Map for storing ISBN and book details.
- (c) Write a Java program using HashMap to:
 - Insert book details
 - Retrieve a book
 - Display all books

(a) Three Suitable Collection

Types Three suitable Java Collection types for an online voting system are:

1. HashSet

- Stores unique voters.
- Prevents the same voter from voting more than once

2. HashMap

- Stores candidate names and their vote counts.
- Provides fast access to vote counts.

2. ArrayList

- Can store and display the list of candidates in a specific order.
- Useful when the order of candidates needs to be maintained.

(b) Structure Using Set and Map We can use:

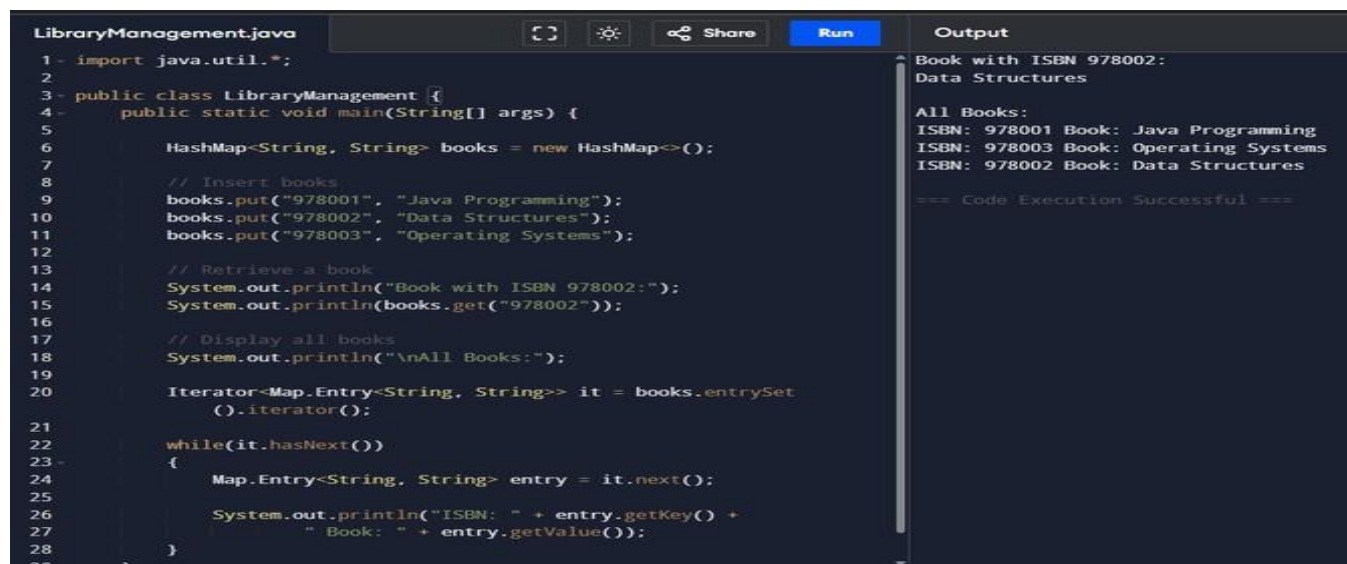
- HashSet<String> → stores voters who have already voted.
- HashMap<String, Integer> →

Example Voted Voters:

HashSet { U101, U102, U103, U104 }

Vote Count:

HashMap { "Ravi" → 2, "Arun" → 3, "Priya" → 1 }



The screenshot shows a Java IDE with a file named `LibraryManagement.java`. The code defines a `LibraryManagement` class with a `main` method. It uses a `HashMap` to store book details (ISBN to title). The code inserts three books, retrieves a book by ISBN, and then iterates over the map to display all books. The output window on the right shows the results of these operations.

```
LibraryManagement.java
1- import java.util.*;
2-
3- public class LibraryManagement {
4-     public static void main(String[] args) {
5-
6-         HashMap<String, String> books = new HashMap<>();
7-
8-         // Insert books
9-         books.put("978001", "Java Programming");
10-        books.put("978002", "Data Structures");
11-        books.put("978003", "Operating Systems");
12-
13-        // Retrieve a book
14-        System.out.println("Book with ISBN 978002:");
15-        System.out.println(books.get("978002"));
16-
17-        // Display all books
18-        System.out.println("\nAll Books:");
19-
20-        Iterator<Map.Entry<String, String>> it = books.entrySet().iterator();
21-
22-        while(it.hasNext())
23-        {
24-            Map.Entry<String, String> entry = it.next();
25-
26-            System.out.println("ISBN: " + entry.getKey() +
27-                               " Book: " + entry.getValue());
28-        }
29-    }
30-}
```

Output

```
Book with ISBN 978002:
Data Structures

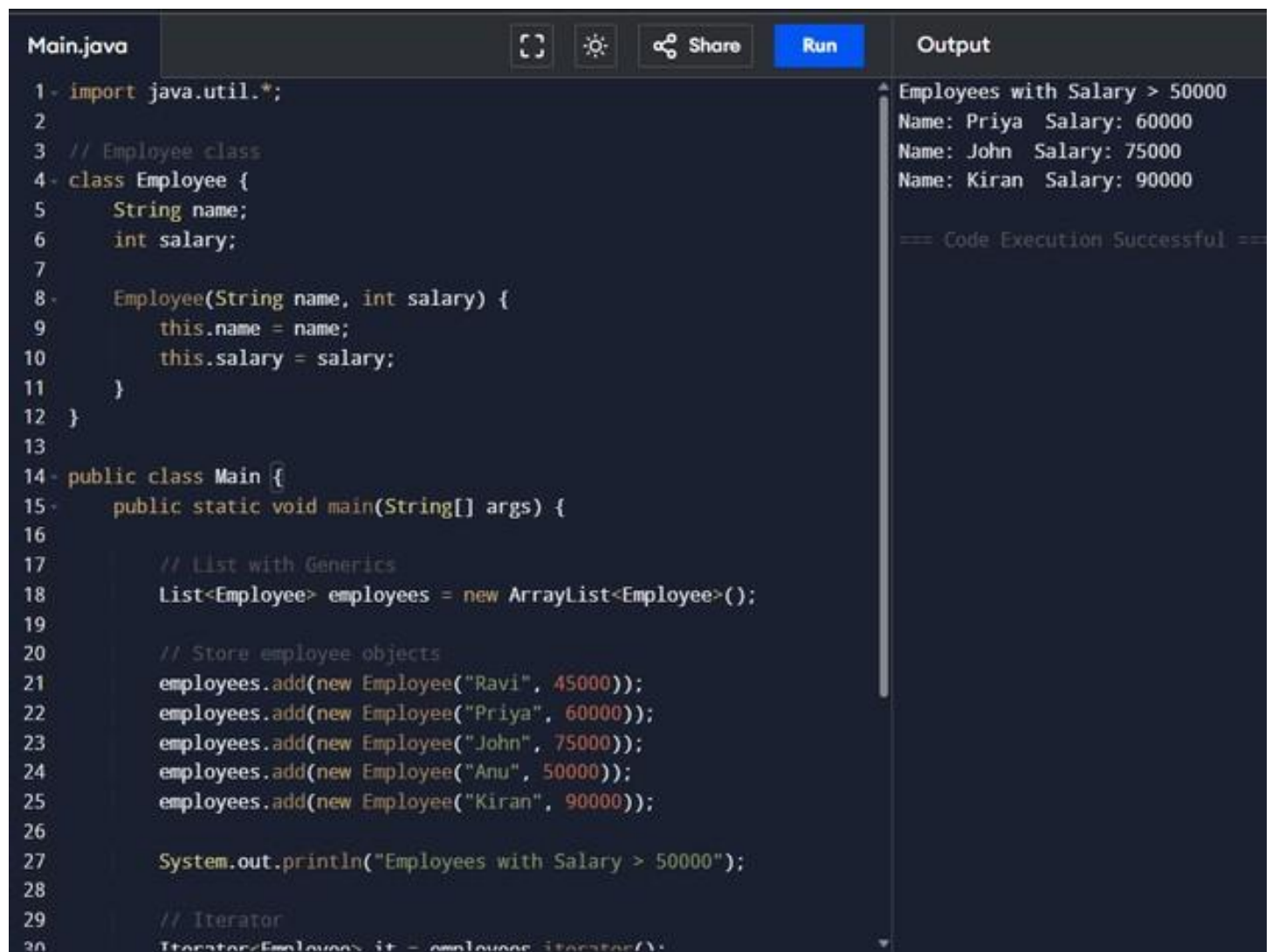
All Books:
ISBN: 978001 Book: Java Programming
ISBN: 978003 Book: Operating Systems
ISBN: 978002 Book: Data Structures

=== Code Execution Successful ===
```

Q4: Employee Data Processing

Suppose that a company processes employee records and filters employees based on salary.

- (a) List three advantages of Generics in Collections.
- (b) Represent how employee data is stored using List with Generics.
- (c) Write a Java program using Iterator to:
 - Store employee objects
 - Display employees with salary > 50,000



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines an 'Employee' class with 'name' and 'salary' attributes, and a 'Main' class that uses a 'List' to store employee objects. The 'Main' class also uses an 'Iterator' to traverse the list and print the names and salaries of employees with a salary greater than 50,000. The output window on the right shows the results of the program execution.

```
1 import java.util.*;
2
3 // Employee class
4 class Employee {
5     String name;
6     int salary;
7
8     Employee(String name, int salary) {
9         this.name = name;
10        this.salary = salary;
11    }
12 }
13
14 public class Main {
15     public static void main(String[] args) {
16
17         // List with Generics
18         List<Employee> employees = new ArrayList<Employee>();
19
20         // Store employee objects
21         employees.add(new Employee("Ravi", 45000));
22         employees.add(new Employee("Priya", 60000));
23         employees.add(new Employee("John", 75000));
24         employees.add(new Employee("Anu", 50000));
25         employees.add(new Employee("Kiran", 90000));
26
27         System.out.println("Employees with Salary > 50000");
28
29         // Iterator
30         Iterator<Employee> it = employees.iterator();
```

Output

```
Employees with Salary > 50000
Name: Priya Salary: 60000
Name: John Salary: 75000
Name: Kiran Salary: 90000

=== Code Execution Successful ===
```

Q5: Online Voting System

Suppose that an online voting system ensures one vote per user and counts votes efficiently.

- (a) Identify three suitable Collection types for this system.

(b) Design a structure using Set and Map for voters and vote counting.

(c) Write a Java program using HashSet and HashMap to:

- Record votes
- Prevent duplicate voting
- Display results

Main.java

Share

Run

```
1 import java.util.*;
2
3 public class OnlineVoting
4 {
5     public static void main(String[] args)
6     {
7         // Store unique voters
8         HashSet<String> voters = new HashSet<>();
9
10        // Vote count
11        HashMap<String, Integer> votes = new HashMap<>();
12
13        // Voting
14        vote("Voter1","Alice",voters,votes);
15        vote("Voter2","Bob",voters,votes);
16        vote("Voter3","Alice",voters,votes);
17        vote("Voter1","Bob",voters,votes); // Duplicate
18
19        System.out.println("\nVoting Results:");
20
21        Iterator<Map.Entry<String,Integer>> it = votes.entrySet()
22            ().iterator();
23
24        while(it.hasNext())
25        {
26            Map.Entry<String,Integer> entry = it.next();
27
28            System.out.println(entry.getKey() + " : " + entry.getValue()
29                ());
30        }
31    }
32 }
```

Output

Voter1 voted for Alice
Voter2 voted for Bob
Voter3 voted for Alice
Voter1 has already voted.

Voting Results:
Bob : 1
Alice : 2

=== Code Execution Successful ===